

High-Performance Linpack Benchmarking on CPU and GPU Clusters: Optimization and Scaling Laws

BIANCA SOLANO, University of New Mexico, USA
ELIUD GARCIA REZA, University of New Mexico, USA
ADAM FASULO, University of New Mexico, USA

This report examines the performance of three computing systems—the Team CPU cluster, the Hopper CPU cluster, and the Easley GPU node—using the High Performance Linpack (HPL) benchmark. Our objective was to measure and optimize each system’s floating-point throughput, expressed in GFLOPS, while gaining practical experience with distributed-memory computation and MPI-based execution.

We first built and configured HPL on the Team Cluster, then systematically tuned key parameters on all three systems, including matrix size (N), block size (NB), and process grid dimensions ($P \times Q$). On Hopper we performed a parameter sweep over NB and process count, identifying a throughput hotspot at $NB = 256$ and 64 processes that reached 2.46 TFLOPS. On the Team Cluster we carried out a strong-scaling study at fixed $N = 40,000$, obtaining a maximum speedup of 7.3 $\times$ and observing clear saturation consistent with Amdahl’s Law.

On the Easley node, equipped with two NVIDIA H100 NVL GPUs, we achieved a peak throughput of 73.1 TFLOPS (about 61% of the two-GPU theoretical peak) and essentially perfect weak scaling when moving from one to two GPUs at constant per-device workload. Kernel-level profiling showed that the benchmark is compute-dominated, with only a small fraction of time spent in communication, and that prominent GFLOPS oscillations are driven by extreme jitter in the main compute kernel and by the alternation between trailing updates and panel broadcasts inherent to the HPL algorithm.

Taken together, these experiments show how practical performance tuning, job scheduling with Slurm, and systematic benchmarking can be used to connect textbook scaling laws to the behavior of real CPU- and GPU-based high-performance computing platforms.

Additional Key Words and Phrases: High Performance Linpack, HPL, high-performance computing, HPC benchmarking, strong scaling, weak scaling, GPU computing, MPI

ACM Reference Format:

Bianca Solano, Eliud Garcia Reza, and Adam Fasulo. 2025. High-Performance Linpack Benchmarking on CPU and GPU Clusters: Optimization and Scaling Laws. 1, 1 (November 2025), 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Authors’ Contact Information: Bianca Solano, University of New Mexico, Albuquerque, NM, USA; Eliud Garcia Reza, University of New Mexico, Albuquerque, NM, USA; Adam Fasulo, University of New Mexico, Albuquerque, NM, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2025/11-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

High-performance computing is in the middle of a transition from homogeneous, CPU-centric systems toward heterogeneous, accelerator-driven architectures. In this changing landscape, evaluating performance requires more than counting theoretical peak FLOPS; it requires understanding how specific CPU and GPU architectures interact with dense linear algebra workloads such as the High Performance Linpack (HPL) benchmark.

In this paper, we present the process and methodology used to run HPL on three systems available to our class: our team cluster, the Hopper CPU cluster, and the Easley GPU node. The primary goal of this project was to evaluate and optimize the computational performance of these systems by measuring their floating-point throughput, expressed in GFLOPS, under realistic memory and runtime constraints.

To accomplish this, we first built and configured HPL on our team cluster and then systematically tuned parameters on all three systems to maximize performance. The key variables we explored were the matrix dimension N , the block size NB , and the process grid dimensions $P \times Q$.

Beyond simply reporting peak GFLOPS, we were also interested in scaling behavior. On the team cluster, we used ... we used strong-scaling experiments (fixed N , increasing process count) to relate our results to Amdahl’s Law [1] and to observe where speedup begins to saturate. On Hopper and Easley, we performed both strong and weak-scaling studies, adjusting N with the number of processes so that the local workload per process remained roughly constant, in the spirit of Gustafson’s Law.

The remainder of this report is organized as follows. In the *Methods* section, we describe the three systems and our tuning strategy for each cluster. The *Results* section then presents our throughput measurements, scaling behavior, and a detailed performance analysis, including an investigation of kernel-level jitter on the Easley node. Finally, we conclude with a discussion of what we learned about practical HPL optimization on both CPU and GPU platforms.

2 Methods

This project aimed to evaluate and optimize the performance of the HPL benchmark on three HPC systems available for the course: the Team Cluster, Hopper, and Easley. All clusters consisted of multiple compute nodes managed through the Slurm workload manager. For each system, we tuned HPL by modifying its configuration file `HPL.dat`, focusing on the matrix size (N), block size (NB), and process grid dimensions ($P \times Q$). We then used these tuned configurations as the basis for strong- and weak-scaling studies.

2.1 System overview

The three systems used in this study differ significantly in CPU generation, memory capacity, and accelerator support, which makes them a useful cross-section of contemporary HPC hardware:

- **Team cluster:** A CPU-only teaching cluster running Rocky Linux 9.3 (Blue Onyx). Each compute node contains two Intel Xeon E5-2670 processors (Sandy Bridge, 8 cores per socket, 16 physical cores total) with simultaneous multithreading enabled (32 hardware threads) and 40 MiB of shared L3 cache across the two sockets. Nodes are provisioned with 62 GiB of main memory and do not provide GPU accelerators. HPL runs here exercise distributed-memory parallelism over a modest number of older-generation CPU cores.
- **Hopper:** A more capable CPU cluster running Rocky Linux 8.10. The Hopper compute nodes used in this work each have two Intel Xeon Gold 6226R processors (Cascade Lake, 16 cores per socket, 32 physical cores total) without hyperthreading (one hardware thread per core), 22.5 MiB of L3 cache per socket, and 92 GiB of RAM. The nodes are configured as two NUMA domains and are connected via Mellanox ConnectX-4 Lx Ethernet and a Mellanox ConnectX-6 InfiniBand adapter. On this system, HPL relies on MPI and optimized CPU BLAS, and our runs used up to two nodes (64 MPI processes) for throughput experiments.
- **Easley:** A GPU-accelerated node running Rocky Linux 9.5 (Blue Onyx). The Easley node used in this study is equipped with two Intel Xeon Gold 6438Y+ processors (Ice Lake, 32 cores per socket, 64 physical cores total), organized into two NUMA domains, and 1.0 TiB of system memory. For acceleration, the node has two NVIDIA H100 NVL GPUs, each with 95,830 MiB of HBM3 memory (compute capability 9.0), managed by an NVIDIA driver at version 570.148.08 with CUDA 12.8. On Easley we used the HPL-NVIDIA 24.09.0 implementation, which offloads dense linear algebra operations to the GPUs and uses NCCL and NVSHMEM for communication.

These configurations allowed us to compare HPL behavior across three regimes: an older, modest CPU teaching cluster (Team), a newer CPU cluster with higher memory bandwidth and modern interconnects (Hopper), and a state-of-the-art GPU-accelerated node (Easley) with high-bandwidth HBM and specialized communication libraries.

2.2 Team Cluster configuration

We began by building and running HPL on our team cluster. HPL was configured to use MPI for inter-process communication and tuned through its input configuration file, `HPL.dat`. We systematically varied the matrix size (N), block size (NB), and process grid dimensions ($P \times Q$).

To maximize floating-point performance (GFLOPS), we conducted an iterative tuning process. First, we selected matrix sizes that used a reasonable amount of the available RAM to maximize compute density without inducing swapping. Block sizes between 32 and 256 were tested to balance cache efficiency against communication overhead. Process grids were chosen to match the number of available cores per node and to maintain an even workload distribution.

This tuning process on the Team Cluster converged to a configuration with a total of 22 MPI processes. The optimal parameters were a matrix size of $N = 40,000$, a block size of $NB = 256$, and a process grid of $P = 2$, $Q = 11$. This 2×11 layout was preferred over a 1×22 layout because it is more square, reducing the communication perimeter of the process grid while still utilizing the majority of available physical cores.

Once we identified a stable, optimized configuration, we performed a strong-scaling study to evaluate cluster performance characteristics.

2.2.1 Strong Scaling Methodology. To illustrate strong scaling and connect our results to Amdahl's Law, we selected a fixed matrix size of $N = 40,000$ and a fixed block size of $NB = 256$. We then conducted a series of HPL runs using 1, 4, 16, and 22 processors, recording execution time and GFLOPS for each configuration. This allowed us to observe how speedup evolves with processor count and to identify the point at which communication overhead and other non-parallelizable costs begin to dominate.

2.3 Hopper Cluster

2.3.1 Throughput Optimization. The Hopper cluster benchmarks were conducted in a parameter-sweeping approach. The benchmarks were performed on 2 nodes, using a large, constant matrix size ($N = 125,000$; $N^2 \times 8 \approx 125$ GB), which spanned approximately 68% of total available memory across the two nodes - for reference, each node had a total memory of 92GB. Block sizes (NB) consisted of increasing powers of 2 from 32 to 1024. P and Q ratios followed a 1:1 ratio for process counts 16 and 64, and a 1:2 ratio for process counts 8 and 32; the process counts (P) varied by increasing powers of 2, from 8 to 64 process. The Hopper node hardware was studied, revealing a 32 cap for processes-per-node, thus, limiting our 2-node test to 64 total processes. An attempt at consistency in extra performance-affecting variables was established by allocating an even amount of processes across the nodes - requiring each node to use half of the total processes used for the benchmark.

2.3.2 Weak Scaling. A Weak-Scaling study was conducted on the **Hopper** cluster to capture the approximate serial, f and parallel, $(1-f)$, fractions of our HPL benchmark. Benchmarks ran on 1 Hopper compute node, testing several processes from 1 up to 32 processes ($p = 1, 2, 3, 4, 8, 12, 16, 24, 32$ to be precise); benchmarks maintained a constant 256 block-size and P to Q ratios that were 1:k ($k = 1, 2, 3$), with the exception of $p = 12$. Due to the benchmarks working with a N^2 problem size, where N is the matrix dimension, N scaled up proportional to $\sqrt{p}$ (i.e. root of the process count) to keep the workload for each process constant (whereas Strong Scaling would allow differing work-per-process as only process counts increase relative to problem size) - starting at 1 process with a dimension of $N = 17320$, with the intention of keeping each process to work with about $3 \cdot 10^8 (\approx 17320^2)$ matrix elements.

2.4 Easley Cluster

We conducted two distinct experiments on the Easley cluster (easley053), a node equipped with two NVIDIA H100 NVL GPUs (94 GiB HBM3 each), using HPL-NVIDIA 24.09.0.

First, for a peak-performance (strong-scaling) test, we tuned parameters to maximize throughput on the 2-GPU setup. This resulted in an optimal configuration of $N = 148,992$ (consuming more than 96% of the available device memory), $NB = 1,536$, and a process grid of $P = 1, Q = 2$.

Second, for a weak-scaling test, we ran a baseline on a single GPU ($N = 99,840$) and a scaled run on two GPUs ($N = 139,776$). These problem sizes were chosen so that each GPU used approximately 80 GiB of memory, keeping the per-device workload roughly constant as the number of GPUs increased from one to two.

For all Easley runs, communication and data movement were optimized by enabling NVIDIA-specific environment variables to tune NCCL-based collectives, use NVSHMEM for GPU-initiated communication, and configure huge pages to reduce TLB pressure. These settings were kept consistent across experiments to isolate the effects of N , NB , and the process grid on overall performance.

To analyze kernel-level behavior, we collected traces using NVIDIA Nsight Systems and grouped kernels into compute, communication (NCCL), and auxiliary categories; these traces form the basis of the time-attribution and jitter analyses presented in the Easley results section.

3 Results

In this section we present the performance results for the three systems under study: the Easley GPU node, the Hopper CPU cluster, and the Team Cluster. We first report weak- and strong-scaling behavior on Easley, then analyze weak scaling and throughput optimization on Hopper, and finally discuss strong scaling on the Team Cluster. Throughout, we connect the observed behavior to the underlying algorithmic and architectural constraints.

3.1 Easley cluster results and analysis

The two experiments on the Easley cluster provide a detailed view of the node's behavior, demonstrating both near-perfect weak scaling and high, but ultimately bounded, peak throughput.

3.1.1 Weak Scaling Results. The weak-scaling experiment demonstrated essentially ideal scaling. The results, comparing a single-GPU run to a two-GPU run with a proportionally larger problem size, are summarized in Table 1.

Table 1. Weak-scaling results on Easley ($k=1$ vs. $k=2$ GPUs). The per-GPU workload is held approximately constant.

Metric	$k = 1$ Run (1 GPU)	$k = 2$ Run (2 GPUs)
Problem Size (N)	99,840	139,776
Memory per GPU	~ 80.3 GiB	~ 79.4 GiB (avg)
Total GFLOPS	36,030	72,160
GFLOPS per GPU	36,030	36,080
Wall Time	18.41 s	25.23 s

Two features of Table 1 are noteworthy. First, the GFLOPS per GPU remain almost identical (36,030 vs. 36,080), indicating that each device sustains essentially the same rate of useful work as we

increase the number of GPUs and the global problem size. Second, the total throughput scales linearly with the number of GPUs. The ideal 2-GPU performance, assuming perfect weak scaling, would be $36,030 \times 2 = 72,060$ GFLOPS. Our measured value of 72,160 GFLOPS corresponds to a weak-scaling efficiency of **100.14%**, which is effectively ideal within measurement noise and suggests that, for this problem size, the 2-GPU configuration behaves almost like a single logical device with twice the resources.

The wall time increases from 18.41 s to 25.23 s as we move from one to two GPUs. This is expected: the larger global problem requires more total work, and the 2-GPU run also incurs additional communication overhead for data exchange over NVLink. Conceptually, this wall time increase is the “cost of parallelization” in a weak-scaling regime. The fact that per-GPU throughput is unchanged while total time increases modestly is consistent with good weak scaling.

3.1.2 Peak performance and Bottleneck analysis. Our optimized peak-performance run on the 2-GPU Easley node achieved a strong result of **73.12 TFLOPS** (73,120 GFLOPS). This was obtained using $N = 148,992$, $NB = 1,536$, and a 1×2 process grid. Given the theoretical peak of roughly 120 TFLOPS for the 2-GPU system, this corresponds to a sustained efficiency of about **61%** of peak, which is competitive for a dense linear algebra benchmark on a real system.

However, as shown in Figure 1, the instantaneous throughput over the course of the run reveals significant, high-frequency performance oscillations, with GFLOPS fluctuating between approximately 79,000 and 70,000.

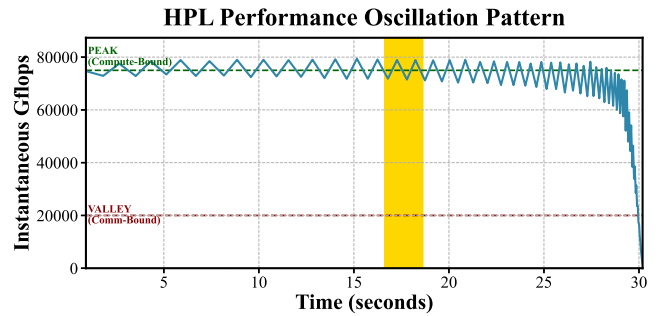


Fig. 1. HPL performance on the Easley 2-GPU node over time, showing alternating peaks and valleys in GFLOPS. The highlighted region (16.6–18.6 s) is analyzed in more detail.

To understand this behavior, we profiled the total execution time by kernel category, as shown in Figure 2. The breakdown reveals that the benchmark is overwhelmingly **compute-dominated (86.2%)**; communication (NCCL) accounts for only about **5.0%** of the total time.

This decomposition suggests that the bottleneck is not simply the *total* amount of time spent in communication, but rather the interaction between computation, panel factorizations, and synchronization. HPL alternates between a compute-bound “trailing update” phase and a more communication-sensitive “panel broadcast” phase during each iteration of the factorization. The oscillatory throughput pattern in Figure 1 mirrors this alternation.

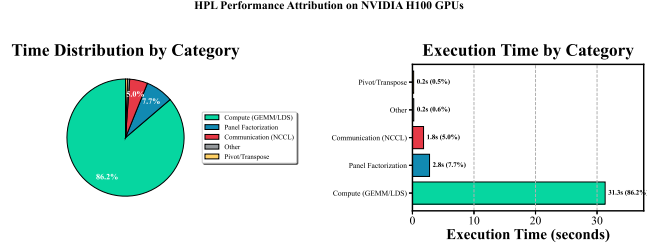


Fig. 2. HPL execution time breakdown on Easley by kernel category. Compute operations (86.2%) dominate the runtime, while communication (5.0%) is a relatively small fraction.

With memory bandwidth and aggregate communication time largely ruled out as primary limiters, our final analysis focused on the *variance* of kernel execution times, or “jitter.” Figure 3 shows a box plot of kernel execution times over the run.

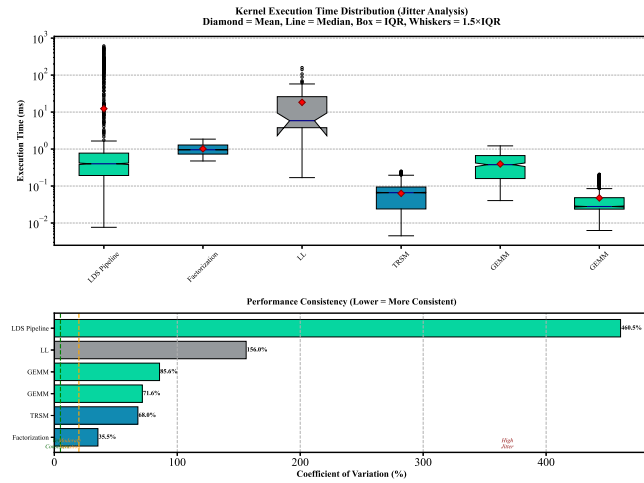


Fig. 3. Kernel execution time variance on Easley. Extreme jitter (coefficient of variation > 460%) in the main compute kernel (kernel_lds_pipeline, bottom) correlates with the observed performance oscillations.

The main compute kernel (kernel_lds_pipeline) exhibits extreme jitter, with a coefficient of variation exceeding 460%. This micro-level variance in compute time is the root cause of the macro-level oscillation in GFLOPS. In other words, the performance trace on Easley is a hardware-level signature of HPL’s algorithmic structure: bursts of highly efficient trailing updates punctuated by synchronization heavy panel operations.

3.2 Hopper cluster results and analysis

3.2.1 Weak scaling analysis. Runtimes were collected; using the Serial Runtime ($p=1$), T_1 , the scaled-speedups ($\text{Speedup}(p) = pT_1/T_p$) were measured and fit to a linear model alongside the process counts - using the SciPy Python Library’s `curve_fit(...)` function. The parameters of the linear fit align with parameters of Guffstafson Law of Speedup, $\text{Speedup}(P) = f + (1 - f)P$, resulting in the following

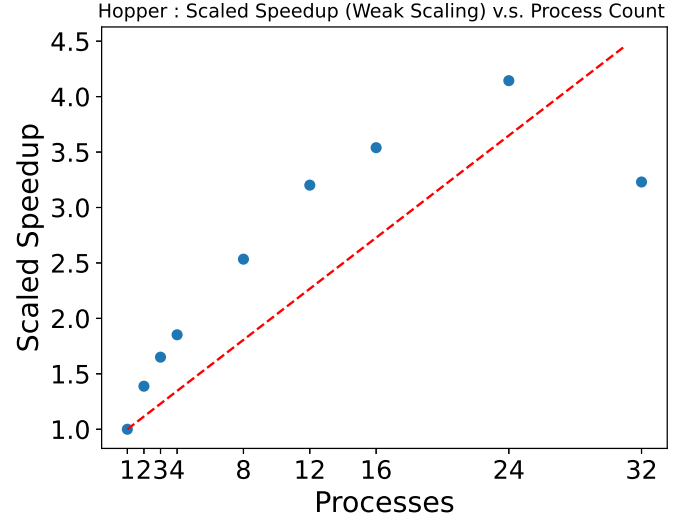


Fig. 4. Linear Model fit to Weak-Scaling study data on single Hopper Node

linear model:

$$\text{Scaled-Speedup} = 0.88482 + 0.11518 P \quad (1)$$

Therefore, the serial fraction of the HPL benchmark is approximately 88.4% and the parallel fraction is approximately 11.5%. Note, the scaled-speedup for 32 processes may be linked to inefficient use of memory; the 32 processes used a problem size such that $N = 97979$ which used ~82% of available node memory - all previous processes used $\leq 64\%$ of the total memory -, thus, promoting swapping and an eventual performance drop. Moreover, the run with 24 processes resulted in the most throughput for the weak-scaling study, with $1.3956e+03$ GFlops.

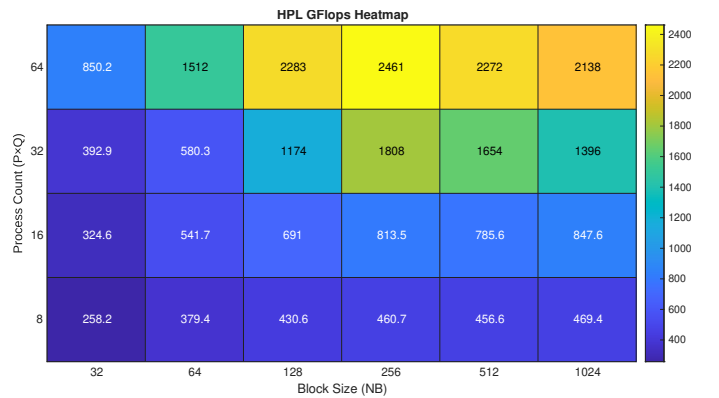


Fig. 5. Heatmap of HPL Benchmarks for Hopper cluster

3.2.2 Throughput optimization. The Optimization study for Hopper demonstrated a throughput “hotspot” that appeared with a block

size of 256 using 64 processes, hence, our team's benchmarks for Hopper maximized throughput at approximately 2.461 TFlops. Evidently, these benchmarks prioritized efficient memory use to reduce swapping - as memory use remained <80% across the two nodes (approximately <= 62% per-node, based on $N=125,000$ and spreading processes evenly across the two nodes). Moreover, this memory efficiency likely led to increased throughput with increasing process counts for each block size. However, as block size increased, lower process counts exhibited a peak at the largest block size (1024), while higher process counts exhibited peak throughput at a block size of 256. This trend to create a "hotspot" around block size of 256 as process count increases is likely due to cache use efficiency, as block sizes are better kept smaller as even more processes need to use them for calculation; with more processes, larger block sizes (like 1024) would result in more frequent cache misses as less blocks can fit in cache than with a size of 256.

3.3 Team Cluster results and analysis

Our testing on the Team Cluster focused on strong scaling—increasing processor count while keeping the global problem size fixed at $N = 40,000$. The results are summarized in Table 2.

Table 2. Strong-scaling results on the Team Cluster ($N = 40,000$, $NB = 256$). Speedup is measured relative to the 1-process run.

Processes	Grid ($P \times Q$)	Time (s)	GFLOPS	Speedup
1	1×1	1746.6	24.4	$1.0\times$
4	2×2	1012.9	42.1	$1.7\times$
16	1×16	322.3	132.4	$5.4\times$
22	2×11	238.8	178.7	$7.3\times$

3.3.1 Strong scaling discussion. The Team Cluster results exhibit the classic features of Amdahl-style strong scaling. As we increase the number of processes from 1 to 22, we do observe significant speedup, but it is far from linear. For example, quadrupling the processes from 1 to 4 yields only a $1.7\times$ speedup, and the jump from 16 to 22 processes improves speedup from $5.4\times$ to $7.3\times$, rather than the ideal $1.375\times$ factor implied by the change in process count.

This non-linear scaling is primarily due to the **surface-to-volume effect**. The total computational work (the "volume," $O(N^3)$) remains fixed, but partitioning the matrix across more processes increases the total inter-process communication (the "surface area," $O(N^2)$). As we push toward 22 processes, each process owns a smaller sub-matrix, and the computation-to-communication ratio degrades. In practical terms, the processors spend an increasing fraction of time exchanging boundary data, synchronizing, and updating distributed panels rather than performing floating-point operations.

These observations are consistent with Amdahl's Law: even if the parallel portion of the code is large, a fixed serial fraction (including communication, synchronization, and local setup) eventually dominates as the process count increases. The Team Cluster experiments therefore provide a concrete example of how strong scaling saturates in a real MPI+HPL setting, even for a relatively well-tuned configuration.

4 Conclusions

This project used the High Performance Linpack benchmark to explore performance and scalability on three different HPC systems: a CPU-based Team Cluster, the Hopper CPU cluster, and the Easley GPU node. By tuning matrix size, block size, and process-grid topology, and by conducting both strong- and weak-scaling studies, we connected textbook scaling laws to the behavior of real hardware and software stacks.

On Easley, we achieved a peak throughput of about 73.1 TFLOPS (roughly 61% of the 2-GPU theoretical peak) and observed essentially perfect weak scaling when moving from one to two H100 GPUs at constant per-device workload. Detailed profiling showed that the benchmark is strongly compute-dominated and that the prominent GFLOPS oscillations arise from extreme jitter in the main compute kernel and the alternation between trailing updates and panel broadcasts in HPL.

On Hopper, a sweep over block size and process count exposed a clear throughput hotspot at $N_B = 256$ with 64 processes. Weak-scaling experiments, interpreted through a Gustafson-style linear fit, highlighted how apparent serial and parallel fractions are affected by memory utilization and runs conducted near the node's memory limits. These results emphasize that good CPU-side HPL performance requires not only sufficient cores but also careful attention to block size, process mapping, and memory headroom.

Finally, strong-scaling experiments on the Team Cluster illustrated the practical limits of Amdahl's Law for a fixed problem size: performance improved as we increased the number of processes, but the speedup curve bent away from linearity as communication and synchronization costs became dominant. Overall, the study reinforced that tuning matters, that memory and communication patterns are as important as raw FLOP/s, and that simple theoretical models remain useful guides when their assumptions and limitations are kept in mind.

5 Author contributions

Bianca Solano implemented and tuned the HPL configuration on the Team Cluster, and wrote the corresponding methods and results sections.

Eliud Garcia Reza conducted HPL benchmarks for the Hopper cluster - including the throughput optimization and weak-scaling study tests. Wrote the methods and results section of the Hopper cluster HPL benchmarks; coded and produced figures for Hopper, Fig.5 and Fig.4 based in the aforementioned tests.

Adam Fasulo conducted the Easley GPU experiments, including both peak-performance and weak-scaling runs, and performed the profiling and kernel-jitter analysis that led to Figures 1, 2, and 3. He also integrated the results from all three systems into a unified narrative and edited the final draft to ensure consistency across sections.

References

- [1] Gene M. Amdahl. 1967. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference (AFIPS '67 (Spring))*. ACM, New York, NY, USA, 483–485. doi:10.1145/1465482.1465560